

Deep Reinforcement Learning

Model-based reinforcement learning

Prof. Dr. Sebastian Peitz

Chair of Safe Autonomous Systems, TU Dortmund

Summer term 2026

Content

Content

- Revisiting the past: Dyna-style algorithms
 - Dyna-Q and Dyna-Q+
 - Prioritized sweeping
- Looking into the future: Planning at decision time
 - MCTS
 - AlphaZero
- Learning a model
 - Dreamer architecture
 - Uncertainty estimation?
 - Learned ensembles

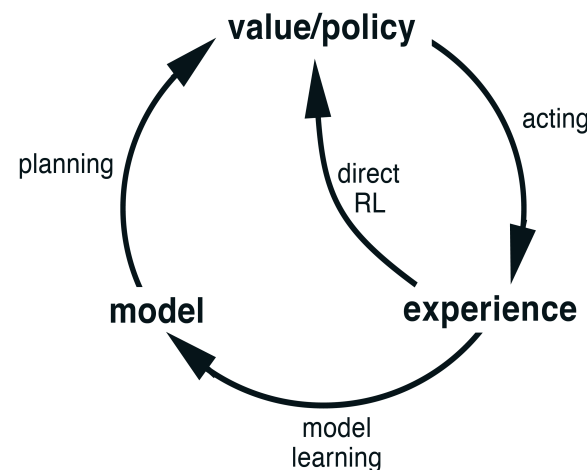
Where are we?

Chapter	Topic	Content
	Basics & tabular methods	
1-5	Bandits, MDPs, Dynamic Programming, Monte Carlo, TD Learning	RL basics in finite dimensions
	Deep-learning-based methods	
6-13	DQN, policy gradients, actor-critic, PPO/SAC, exploration	Deep RL from basics to modern algorithms
	Model-Based Control	
14	Optimal control & feedback control	How to do control when we know the model
15	Model-based reinforcement learning	RL with known and learned models
	Advanced Topics	

Lecture contents

Learning vs. planning in the RL context

- *Learning* and *planning* are closely related in RL (Sutton and Barto 1998).
- The key difference is the *origin of the training data*:
 - **Learning**: real experience,
 - **Planning**: simulated experience.



- Simplest example: *Q*-learning vs. *Q*-planning:

Algorithm: *Q*-learning

for $t = 0, 1, 2, \dots$

Sample action $a_t \sim \pi(s_t)$

Observe (r_t, s_{t+1}) **from environment**

Update Q given (s_t, a_t, r_t, s_{t+1}) :

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]$$

Update policy $\pi = \epsilon\text{-greedy}(Q)$

Algorithm: *Q*-planning

for $t = 0, 1, 2, \dots$

Sample action $a_t \sim \pi(s_t)$

Use model to create sample (r_t, s_{t+1})

Update Q given (s_t, a_t, r_t, s_{t+1}) :

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]$$

Update policy $\pi = \epsilon\text{-greedy}(Q)$

Unified view: create experience using a model $\Rightarrow$ model $\longrightarrow$ simulated experience $\xrightarrow{\text{backups}}$ values $\longrightarrow$ policy

Revisiting the past: Dyna algorithms

The Dyna- Q concept

Easiest way to create a model: Remember past $a, s \rightarrow s'$ and r .

Algorithm: Dyna- Q (Tabular RL)

loop forever:

Sample action a ϵ -greedily using the current Q estimate

Take action a , observe reward r and next state s'

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{\hat{a}} Q(s', \hat{a}) - Q(s, a)]$$

Add transition to our **model**

for $i = 1, \dots, n$:

$s \leftarrow$ random previous state

$a \leftarrow$ random previous action

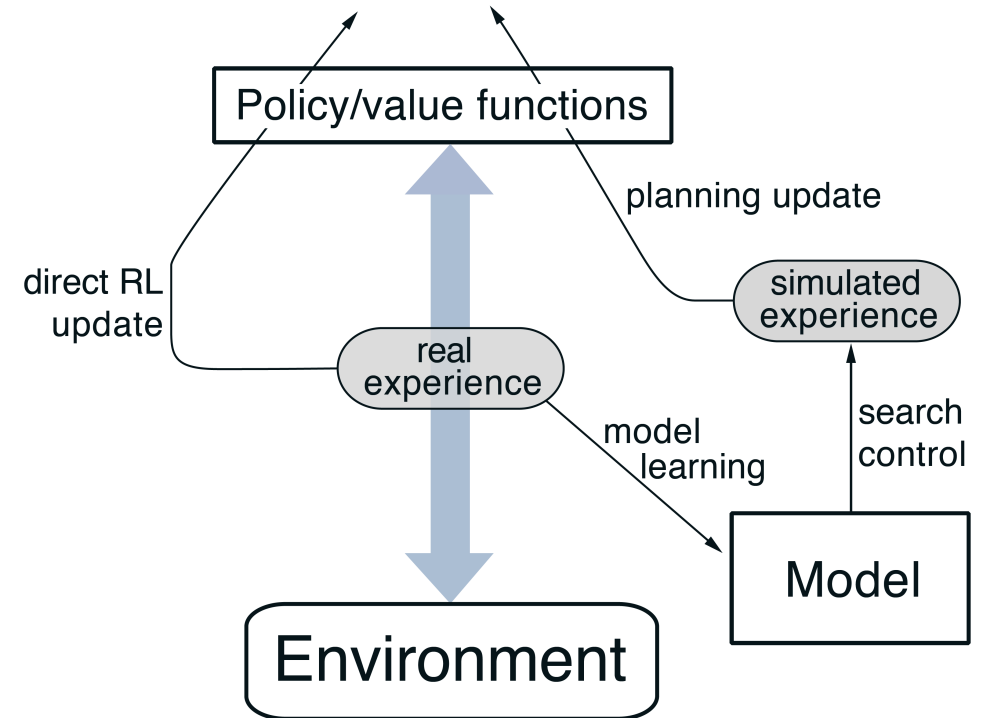
$r, s' \leftarrow \text{model}(s, a)$

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{\hat{a}} Q(s', \hat{a}) - Q(s, a)]$$

Why does this even make sense? Aren't we just doing the exact same calculation?

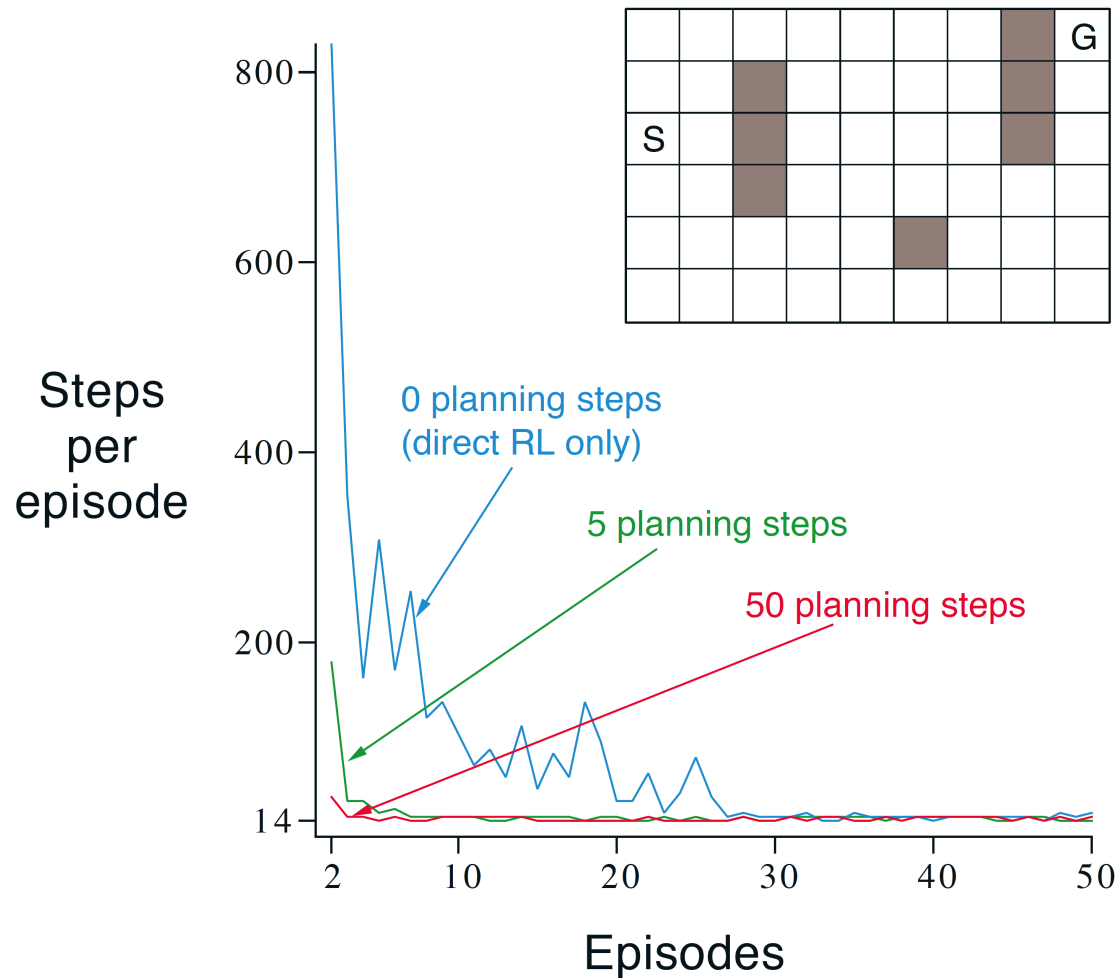
$\Rightarrow$ our approximation of $Q(s, a)$ may be better later on!

$\Rightarrow$ sparse rewards get "time to travel" through the environment!



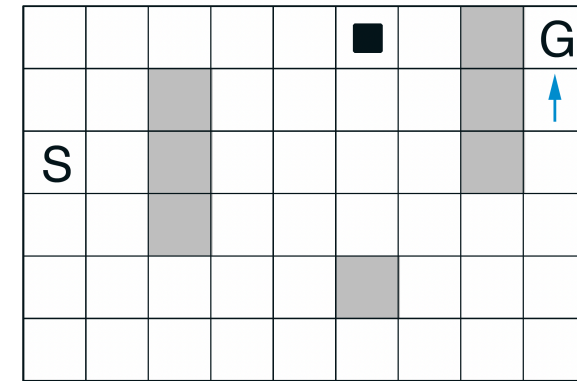
The general Dyna Architecture. Real experience, passing back and forth between the environment and the policy, affects policy and value functions in much the same way as does simulated experience generated by the model of the environment (Source: (Sutton and Barto 1998, Fig. 8.1)).

Example: Dyna Maze (1)

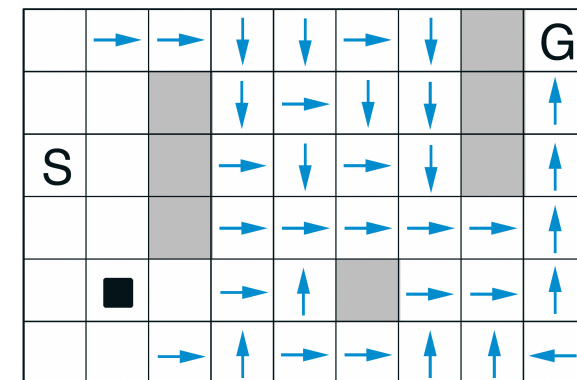


A simple maze (inset) and the average learning curves for Dyna-Q agents varying in their number of planning steps (n) per real step. The task is to travel from S to G as quickly as possible (Source: (Sutton and Barto 1998, Fig. 8.2)).

WITHOUT PLANNING ($n=0$)



WITH PLANNING ($n=50$)



Policies found by planning and nonplanning Dyna-Q agents halfway through the second episode. The arrows indicate the greedy action in each state; if no arrow is shown for a state, then all of its action values were equal. The black square indicates the location of the agent (Source: (Sutton and Barto 1998, Fig. 8.3)).

When the model is wrong

In the previous maze example, the environment was perfect and the transitions deterministic.

Reasons for incorrect or inaccurate memories:

- The environment is stochastic and only a limited number of samples have been observed.
- The model was learned using function approximation that has generalized imperfectly.
- The environment has changed and its new behavior has not yet been observed.

Can we recover from imperfect memories and adapt?

+ If the old model is optimistic, there is a good chance!

— If the old model is pessimistic, it is very hard to find new opportunities!

- even with ϵ -greedy, it is very unlikely to randomly sample a sequence that ends up finding the better solution.
- the old **exploration-exploitation dilemma!**

Dyna-Q+

Two changes can help our simple Dyna-Q algorithm to foster exploration.

1. For each state, *count* the time τ since it has been revisited last (where $\kappa > 0$ is a hyperparameter) and augment the reward by a bonus:

$$r + \kappa\sqrt{\tau}.$$

- The longer we have not revisited a state-action pair s, a , the larger the (boosted) reward for revisiting.

2. In the **planning phase**, we are allowed to try out new actions, even if we have not used them before.

- Since we cannot know the next state s' or the reward r in such a case, the initial setting is $s' = s$ and $r = 0$.
- Nevertheless, not trying out new actions results in them getting large τ and thus, large augmented rewards! $\Rightarrow$ Update towards higher Q estimate.
- Eventually, the **learning phase** will actually explore the new action.
 - If it truly is better: $\tau \leftarrow 0$, but confirmation by large r .
 - If it was optimistic: $\tau \leftarrow 0$, and rejection due to small r .

Algorithm: Dyna-Q (Tabular RL)

loop forever:

(*Learning / Experience*)

Sample action a ϵ -greedily

Take action a , observe r and s'

$Q(s, a) \leftarrow Q(s, a)$

$+ \alpha [r + \gamma \max_{\hat{a}} Q(s', \hat{a}) - Q(s, a)]$

Add transition to our **model**

(*Planning*)

for $i = 1, \dots, n$:

$s \leftarrow$ random previous state

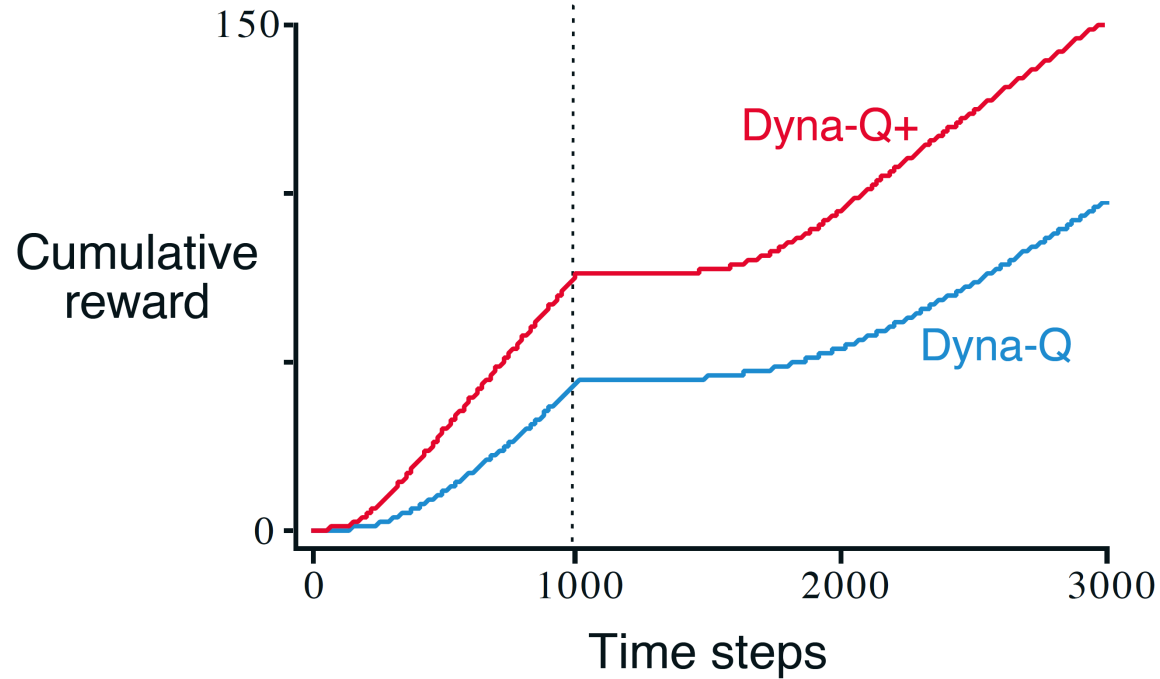
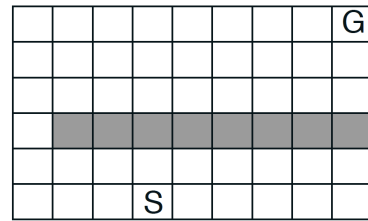
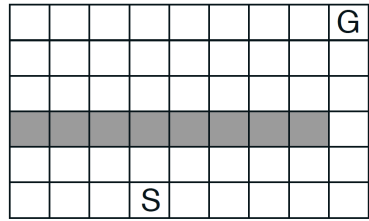
$a \leftarrow$ random previous action

$r, s' \leftarrow \text{model}(s, a)$

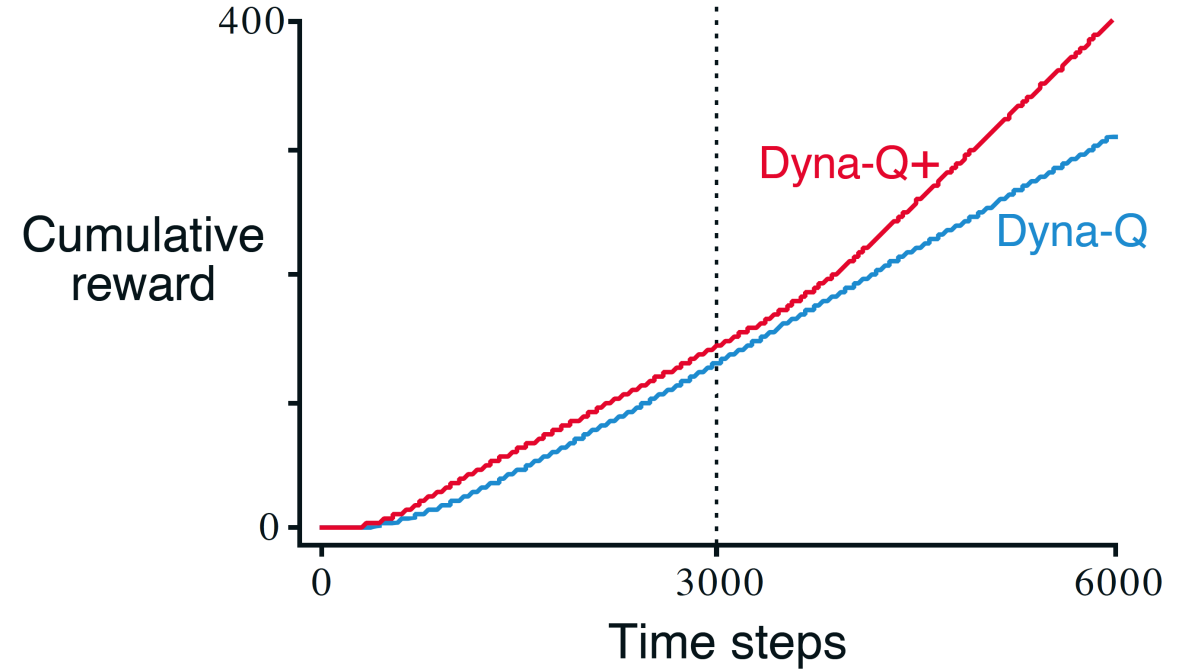
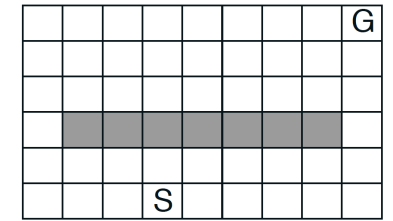
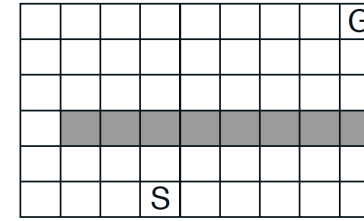
$Q(s, a) \leftarrow Q(s, a)$

$+ \alpha [r + \gamma \max_{\hat{a}} Q(s', \hat{a}) - Q(s, a)]$

Example: Dyna Maze (2)



Average performance of Dyna agents on a blocking task. The left environment was used for the first 1000 steps, the right environment for the rest. Dyna-Q+ is Dyna-Q with an exploration bonus that encourages exploration (Source: (Sutton and Barto 1998, Fig. 8.4)).



Average performance of Dyna agents on a shortcut task. The left environment was used for the first 3000 steps, the right environment for the rest (Source: (Sutton and Barto 1998, Fig. 8.5)).

Prioritized sweeping

Algorithm: Dyna-Q (Tabular RL)

initialize: $Q(s, a)$, $\text{model}(s, a)$ for all s, a ,
empty queue $\mathcal{Q}$, threshold $\xi > 0$.

loop forever:

$a \sim \pi(\cdot | s, Q)$ (e.g. ϵ -greedy)

Take action a , observe reward r and next state s'

Add transition to our **model**

$P \leftarrow [r + \gamma \max_{\hat{a}} Q(s', \hat{a}) - Q(s, a)]$

if $P > \xi$ then insert (s, a) into $\mathcal{Q}$ with priority P

loop: Repeat n times while $\mathcal{Q}$ is not empty

$(s, a) = \arg \max_P \mathcal{Q}$

$r, s' \leftarrow \text{model}(s, a)$

$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{\hat{a}} Q(s', \hat{a}) - Q(s, a)]$

for $\forall (\bar{s}, \bar{a})$ predicted to lead to s :

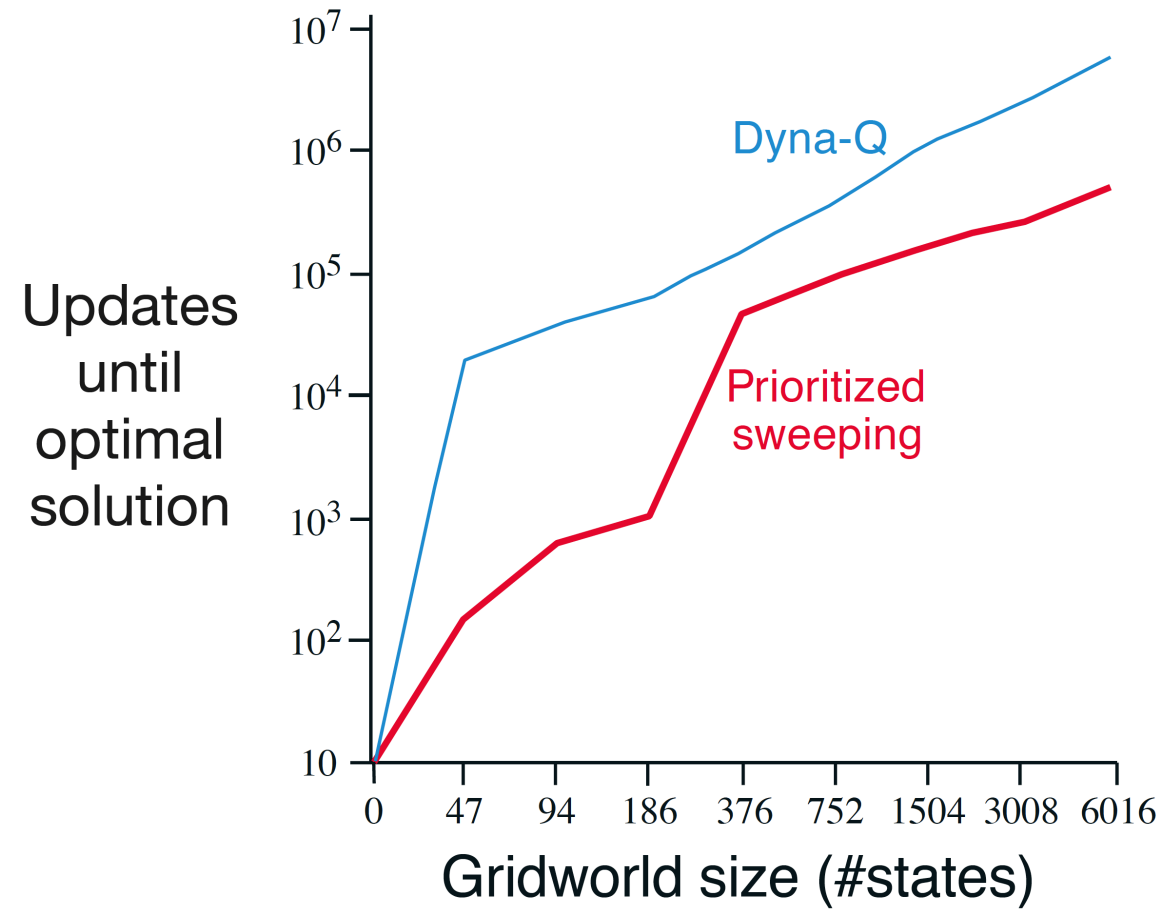
$\bar{r} \leftarrow$ predicted reward for $(\bar{s}, \bar{a}, s)$

$P \leftarrow [\bar{r} + \gamma \max_{\hat{a}} Q(s, \hat{a}) - Q(\bar{s}, \bar{a})]$

if $P > \xi$ then insert $(\bar{s}, \bar{a})$ into $\mathcal{Q}$ with priority P

- Dyna-Q (randomly) samples from the memory buffer.
 - Many planning updates maybe pointless, e.g., zero-valued state updates during early training.
 - In large state-action spaces: inefficient search since transitions are chosen far away from optimal policies.
- Better: focus on important updates.
 - In episodic tasks: **backward focusing** starting from the goal state.
 - In continuing tasks: **prioritize** according to impact on value updates.
- Solution method is called **prioritized sweeping**.
 - Build up a queue of every state-action pair whose value estimate would change significantly.
 - Prioritize updates by the size of change.
 - Neglect state-action pairs with only minor impact.

Example: Dyna Maze (3)



Prioritized sweeping can dramatically increase the speed at which optimal solutions are found in maze tasks such as this one. These data are for a sequence of maze tasks of exactly the same structure as the first one where we studied varying n , except that they vary in the grid resolution. Both systems made at most $n = 5$ updates per environmental interaction (Source: (Sutton and Barto 1998, Example 8.4)).

Looking into the future: Planning at decision time

Planning at decision time

What we have discussed so far: Background planning

- Gradually improves policy or value function if time is available.
- Backward view: re-apply gathered experience.
- Feasible for fast execution: policy or value estimate are available with low latency (important, e.g., for real-time control).

Alternative use of a model: Planning at decision time

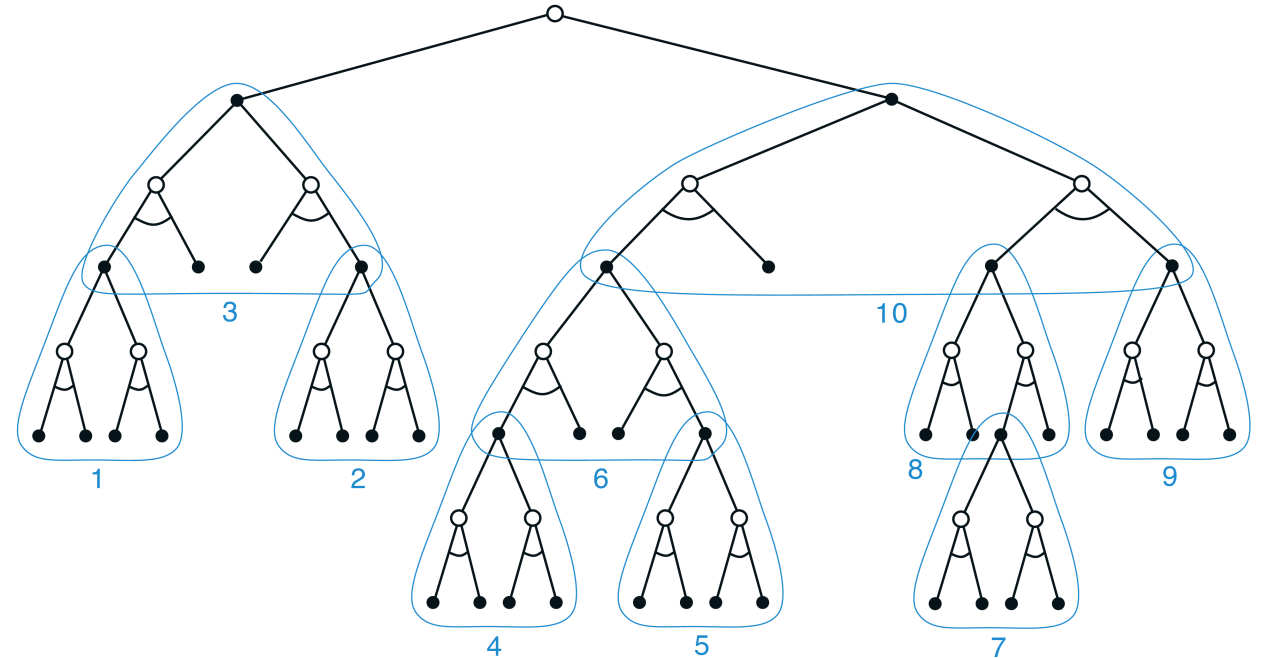
- Select single next future action through planning.
- Forward view: predict future trajectories starting from current state.
- Typically discards previous planning outcomes (start from scratch after state transition).
- If multiple trajectories are independent: easy parallel implementation.
- Most useful if fast responses are not required (e.g., turn-based games or slow systems).

Question: Have we already seen planning algorithms?

- Yes! Optimal control and model predictive control (MPC) fall into this category.
- Now: How to use planning and learning at the same time.
- But first: planning in discrete settings.

Heuristic search

- If we have a model, we can explore all possible state-action sequences from our current state s .
- Challenge: the *curse of dimensionality*. With an increasing prediction horizon, the number of options increases exponentially!
 - For example, let's assume that we have $|\mathcal{A}| = 3$ choices in each step.
 - First prediction step: **3** choices.
 - Second prediction step: **9** choices.
 - Tenth prediction steps: $3^{10} = 59,049$ choices.



Similar to dynamic programming, Heuristic search can be implemented as a sequence of one-step updates (shown here outlined in blue) backing up values from the leaf nodes toward the root. The ordering shown here is for a selective depth-first search (Source: (Sutton and Barto 1998, Fig. 8.9)).

Heuristic search:

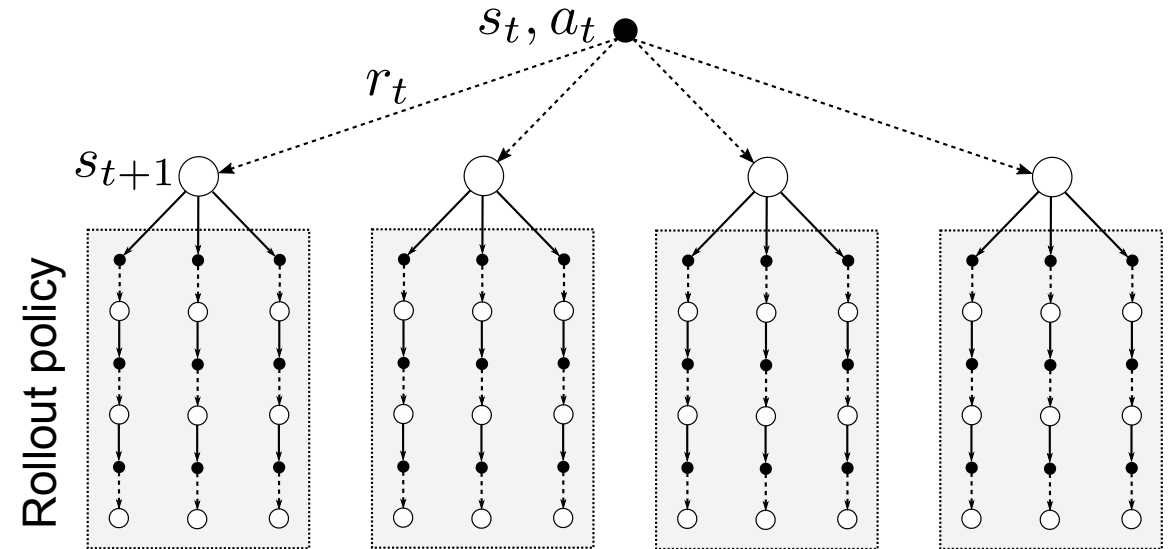
- Stop exploration of the tree at some point and replace neglected sequence by some hand crafted heuristic.
- The heuristic has to be very fast such that we can play assess many states and go deep into the tree.
- Drawback: Inaccurate if the heuristic is bad.

- It can be very hard to find a good heuristic.

Rollout algorithms

Alternative to a heuristic?

- If finding a heuristic is hard, we can instead use a so-called **rollout policy** π^r .
- π^r doesn't have to be very good, but fast to evaluate.
- Approach: take all possible actions a in s , then for each, follow some rollout policy π^r to the end many times.
- Theoretical argument: policy improvement theorem.
 - If π^r is the same for all experiments and the value for one action is higher than for another, then the policy is better according to the PIT.



Simplified processing diagram of rollout algorithms (Adapted from [Abdelwanis et al. 2026](#), Fig. 7.12)).

Question: Which function does the rollout serve?

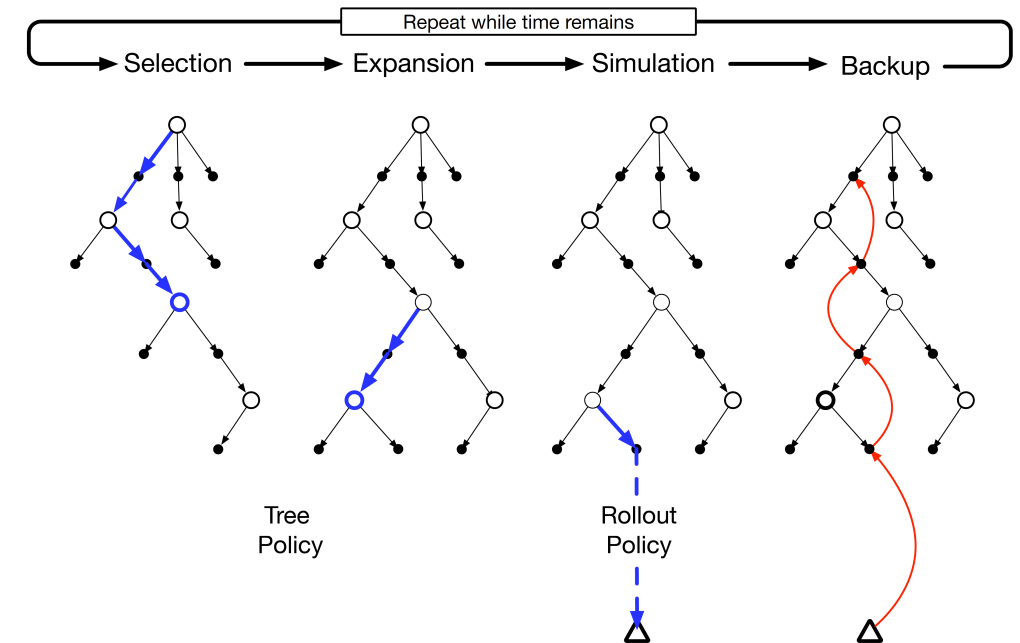
- We have seen a very similar structure in temporal difference learning: The TD target is $y_t = r_t + \gamma V(s_{t+1})$.
- The rollout yields a Monte-Carlo estimate of the value along the branches of the decision tree.

Monte-Carlo Tree Search (MCTS)

- Let's explore MCTS in the special setting of playing games: $r = 1$ if a game is won; $r = 0$ otherwise.
 - We can rate an action by the winning chance (number of wins divided by number of games played with this starting action).

Monte-Carlo Tree Search (MCTS)

- Selection** Starting at the root s , navigate down the existing tree by choosing the *most promising* nodes.
 - Upper Confidence Bound* to balance exploration/exploitation.
 - Move down the tree until a node isn't fully expanded yet.
- Expansion** Unless terminal, create new child node(s).
- Simulation** Fast *rollout* until win/loss $\Rightarrow r = 1 / r = 0$.
- Backpropagation** Pass r back through all nodes visited during selection phase and update node statistics ($\#$ wins / $\#$ visits).



Monte Carlo Tree Search (Source: (Sutton and Barto 1998, Fig. 8.10)).

Execution

- Many MCTS runs: tree becomes highly asymmetric.
 - Deep exploration of paths that look promising.
- When the thinking time is up:
 - Compare immediate children of the root node.
 - Choose the one that was visited the most.
- After opponent move, start MCTS again from the new state.

- Little time on paths that lead to quick losses.
- Make that move in the real world.

MCTS details

- To guide this asymmetric exploration, MCTS relies on a concept from probability theory known as the Multi-Armed Bandit problem. Imagine you are in a casino facing a row of slot machines (multi-armed bandits), and you want to maximize your payouts. Do you stick with the machine that has paid out decently so far (exploitation), or do you try a different machine that you haven't played much yet (exploration)?
- MCTS treats every choice point in the tree as a multi-armed bandit. To solve it, it uses a formula called UCB1 (Upper Confidence Bound). Or also

$$UCT = \frac{w_i}{n_i} + c \sqrt{\frac{\ln N}{n_i}}$$

- During the Selection phase, whenever MCTS needs to choose between different child nodes, it calculates the UCB1 value for each option and picks the one with the highest score. Here is the formula:

$$UCB1 = \frac{w_i}{n_i} + c \times \sqrt{\frac{\ln N_i}{n_i}}$$

Breaking Down the Formula The formula is a beautiful tug-of-war between two distinct forces: UCB1 Score = [Exploitation

Component] + [Exploration Component]

The Exploitation Term ($\frac{w_i}{n_i}$): w_i is the number of wins simulated through child node i so far. n_i is the number of times child node i has been visited. What it does: This is simply the win rate of the node. If a move has won 80 out of 100 simulations, its win rate is 0.8. The higher the win rate, the more attractive this node is.

The Exploration Term ($\sqrt{\frac{\ln N_i}{n_i}}$): N_i is the total number of times the parent node has been visited. $\ln N_i$ is the natural logarithm of that parent visit count. n_i is (again) the number of times this specific child node has been visited. What it does: This measures how much we don't know about this move. Notice that n_i is in the denominator. If we rarely visit a node, n_i is very small, which makes this entire exploration term grow quite large. It acts as a “bonus” for neglected moves.

The Exploration Constant (c): This is a parameter you can tune (theoretically, it is mathematically proven to be $\sqrt{2}$, but in practice, developers adjust it). What it does: It acts as a volume knob. Increase c , and the AI becomes adventurous, searching wildly new ideas. Decrease c , and the AI becomes conservative, heavily focusing only on its best-performing moves.

How the Balancing Act Works in Practice

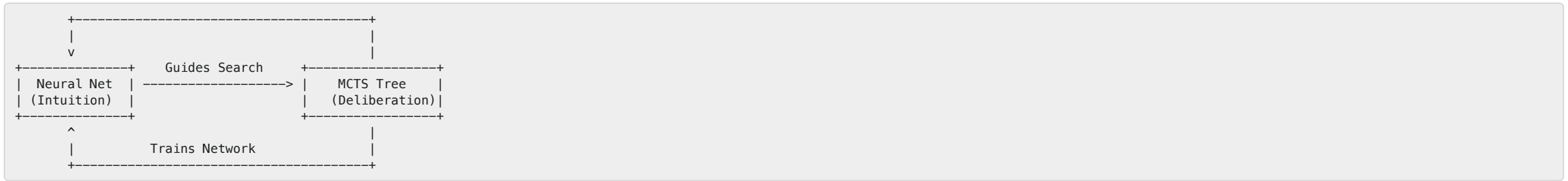
Imagine a node that looks terrible at first—maybe it lost its first 2 simulations. Its exploitation score is 0. MCTS will start ignoring it to focus on better moves. However, as MCTS simulates those other better moves thousands of times, the parent visit count (N_i) grows massive. Because of that $\ln N_i$ in the numerator, the Exploration Term for our ignored node steadily climbs. Eventually, the exploration bonus becomes so high that it overrides the 0% win rate. MCTS is forced to say, “Okay, it’s been a while, let me double-check that ‘bad’ move just in case I missed a brilliant trap.” If the move fails again, its visit count (n_i) goes up, which immediately shrinks the exploration bonus back down. If it turns out to be a hidden masterpiece, its win rate starts climbing, and MCTS naturally incorporates it into its favorite strategies.

Example: AlphaZero

The AlphaZero Loop: How the Tree Teaches the Network

In traditional reinforcement learning, an agent learns by interacting directly with the environment. But in AlphaZero-style algorithms, the agent uses **MCTS as an internal simulator** to generate high-quality data.

The genius of AlphaZero is that the neural network and the MCTS tree exist in a symbiotic loop: **The network guides the tree search, and the tree search trains the network.**



Here is exactly how this training process works, step-by-step.

Step 1: Self-Play and “Thinking Time”

AlphaZero plays millions of games against itself. For every single move in a game, it pauses and runs an MCTS search.

Let’s say the neural network is currently a bit weak. It looks at the board and thinks, “*Move A looks okay, maybe a 50% chance of winning.*” However, MCTS doesn’t just rely on the network’s initial reaction. It runs thousands of simulations, looks several moves ahead, and discovers a hidden tactical blunder. After the search finishes, MCTS concludes, “*Actually, Move A is terrible. Move B is much better.*”

Step 2: The Tree as a Teacher (Policy Improvement)

Because MCTS actually “thought” about the future, its final decision is much smarter than the neural network’s initial guess. MCTS has essentially upgraded the agent’s knowledge.

We can represent the MCTS results as a probability distribution based on how many times each move was visited. Let’s call this target distribution π (pi).

- The neural network initially guessed: [Move A: 70%, Move B: 30%]
- After searching, MCTS visited: [Move A: 10%, Move B: 90%] (π)

We save this board state along with the MCTS recommendation (π) into a massive data bank of training examples.

Step 3: The Game Outcome (Value Estimation)

The agent keeps playing using MCTS until the game naturally ends. Let’s say the agent eventually wins. We go back to every single board state visited during that game and label it with the final outcome: $z = +1$ (a win).

Now, we have a perfect training dataset for our neural network consisting of three things:

1. The raw board state.
 2. The smart MCTS move recommendations (π).
 3. The actual, final game outcome (z).
-

Step 4: Updating the Brain (The Loss Function)

At the end of the day, we take the neural network and train it on this dataset. The network outputs two things for any board state: a move probability vector p , and a predicted win value v .

We force the network to change its internal weights using a dual-purpose loss function:

$$\text{Loss} = (z - v)^2 - \pi \log p + c||\theta||^2$$

Let's break down what this math is forcing the network to learn:

- $(z - v)^2$ (**The Value Loss**): This minimizes the difference between the network's prediction (v) and who actually won the game (z). It teaches the network to look at a board and accurately predict, *"Am I winning or losing?"*
- $-\pi \log p$ (**The Policy Loss**): This forces the network's raw intuition (p) to match the smart, deeply calculated MCTS search results (π). It teaches the network, *"Next time you see a similar board, don't waste time thinking about Move A. Make your initial guess look more like Move B."*
- $c||\theta||^2$ (**Regularization**): A standard machine learning penalty to prevent the network from overfitting or memorizing specific games.

The Result: Exponential Growth

Once the network is updated, you start the self-play process over again.

Because the network's raw intuition is now smarter, the next round of MCTS searches can start from a much higher baseline, allowing it to search even deeper and discover even more complex strategies. Through this continuous cycle of self-improvement, the system rapidly evolves from playing completely random moves to achieving superhuman grandmaster status without ever looking at a human game.

Learning a model

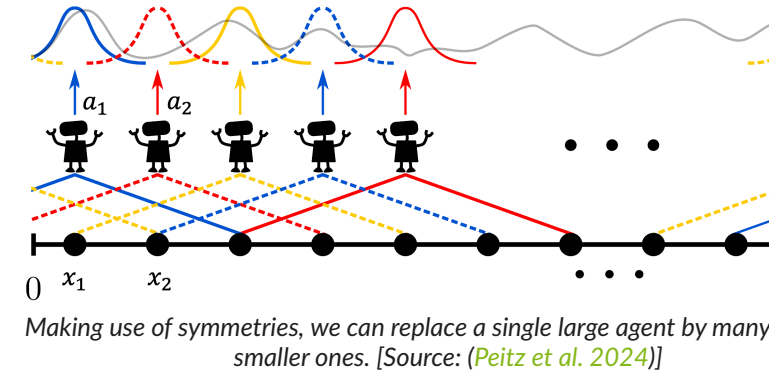
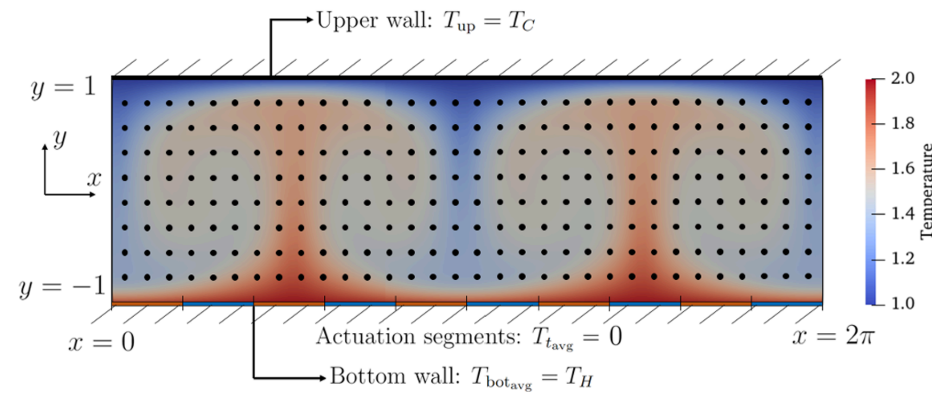
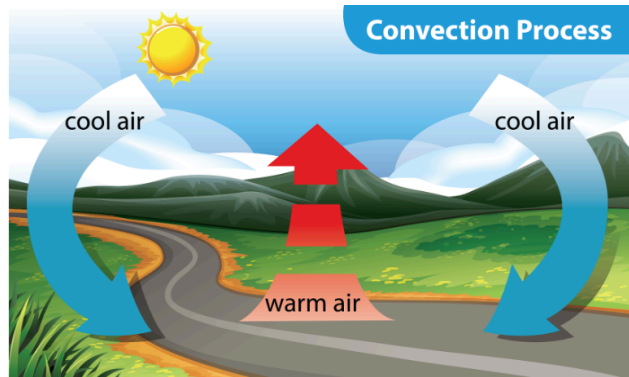
Surrogate modeling

aka World models

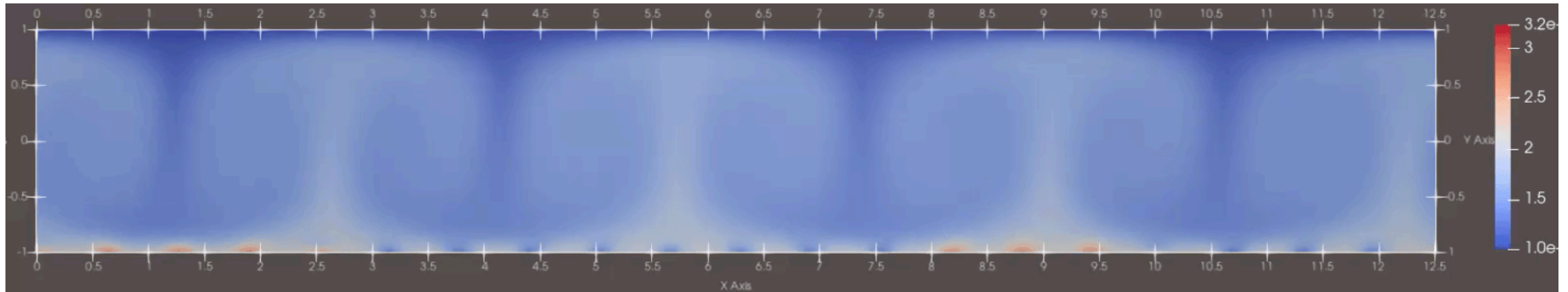
Challenges & questions regarding world modeling

- Data acquisition
 - Which data should we use?
 - How much data?
 - Collected under which actions?
- The policy changes the state distribution: ρ_{π} .
- Intertwining RL and model learning?

Example: Rayleigh-Bénard convection (1)

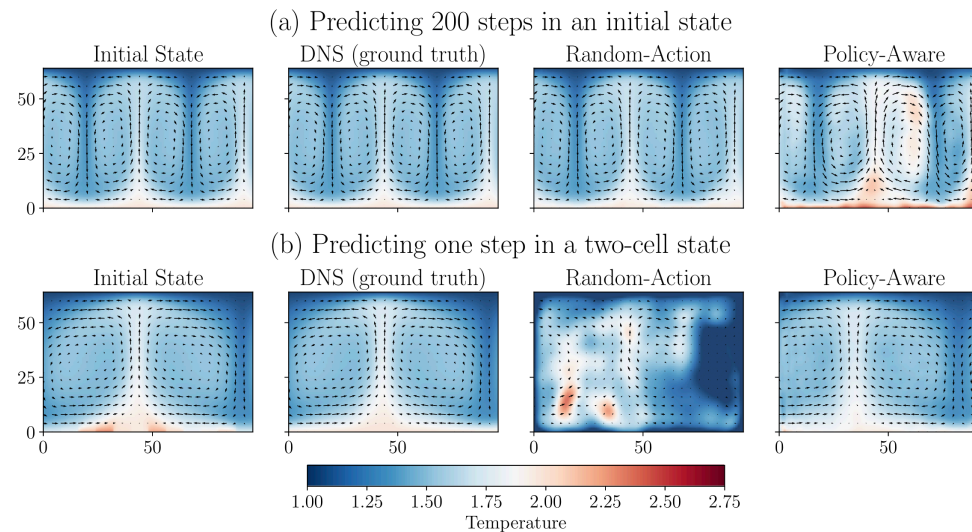
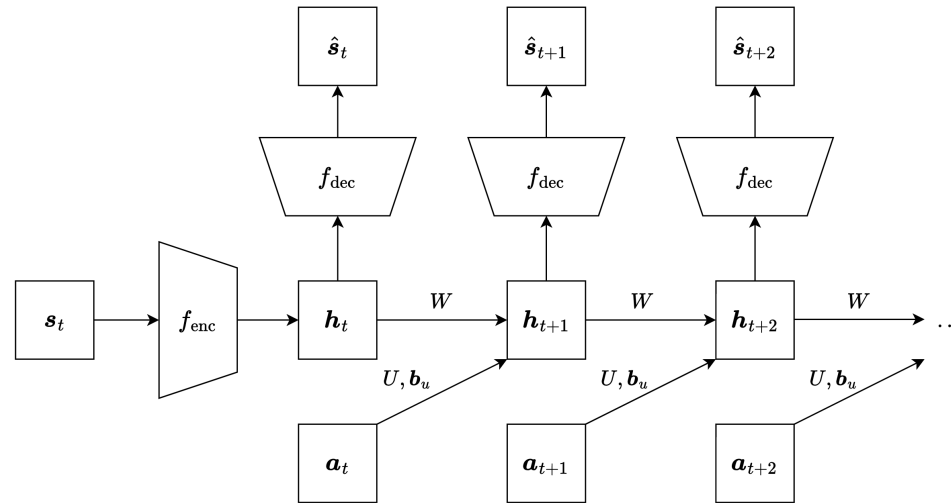


Multi-agent RL → merging of convection cells reduces the convective heat transport

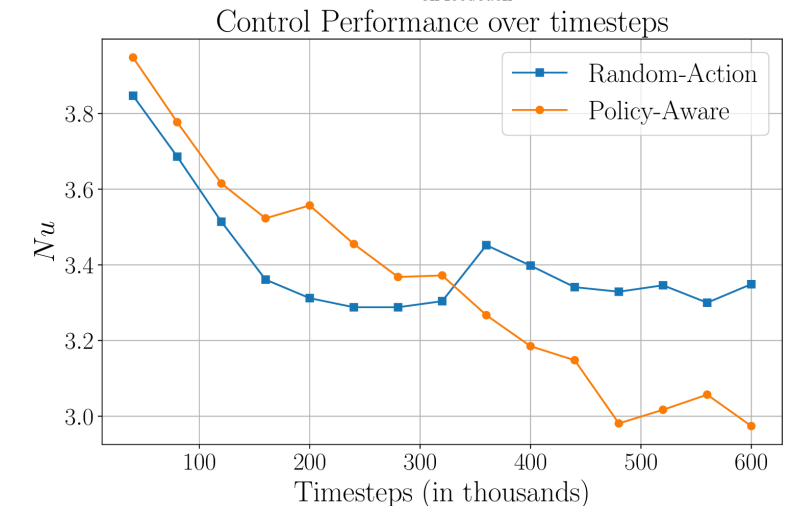
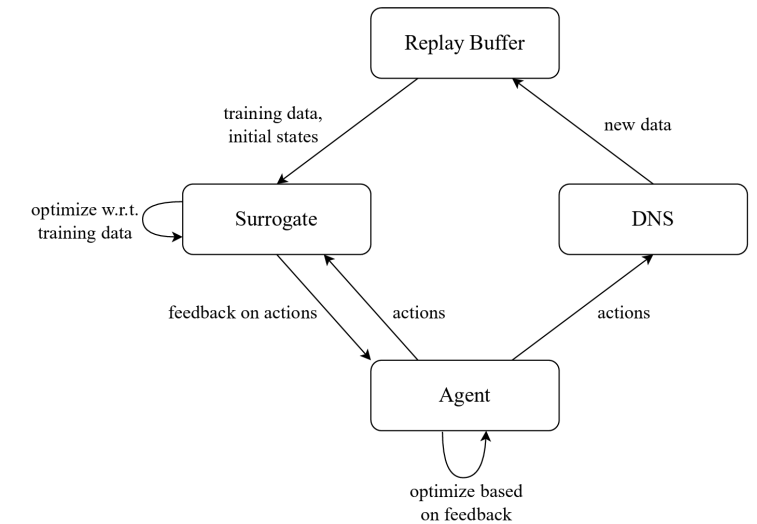


Example: Rayleigh-Bénard convection (2)

- World model:
Autoencoder plus linear model in latent space.
- Intertwined world modeling and policy learning.
- Strictly required for ensuring model performance.
 - Otherwise, prediction before / after cell merging is impossible to predict (see bottom left).
 - Policy-aware data collection strongly improves RL



Results from (Plotzki and Peitz 2026)



The dreamer architecture

Data-driven MPC

Uncertainty estimation

Ensembles

References

- Abdelwanis, Ali, Barnabas Haucke-Korber, Darius Jakobeit, Wilhelm Kirchgässner, Marvin Meyer, Maximilian Schenke, Hendrik Vater, Oliver Willsch, and Daniel Weber. 2026. "Reinforcement Learning: A Comprehensive Open-Source Course." *Journal of Open Source Education* 9 (97). The Open Journal: 306. doi:[10.21105/jose.00306](https://doi.org/10.21105/jose.00306).
- Peitz, Sebastian, Jan Stenner, Vikas Chidananda, Oliver Willsch, Steven L. Brunton, and Kunihiro Taira. 2024. "Distributed Control of Partial Differential Equations Using Convolutional Reinforcement Learning." *Physica D: Nonlinear Phenomena* 461: 134096.
- Plotzki, Tim, and Sebastian Peitz. 2026. "Koopman-Based Surrogate Modeling for Reinforcement-Learning-Control of Rayleigh-Benard Convection." *arXiv:2603.28074*.
- Sutton, R. S., and A. G. Barto. 1998. *Reinforcement Learning: An Introduction*. MIT Press.